SPEAR-JSON: Selective parsing of JSON to enable accelerated stream processing on FPGAs

Tobias Hahn, Stefan Wildermann, Jürgen Teich
Chair for Hardware/Software Co-Design, Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU), Germany
<tobias.hahn,stefan.wildermann,juergen.teich>@fau.de

2023 33rd International Conference on Field-Programmable Logic and Applications (FPL) | 979-8-3503-4151-5/23/$31.00 ©2023 IEEE | DOI: 10.1109/FPL60245.2023.00034

***Abstract*—Big Data applications frequently involve the processing of data streams encoded in semi-structured data formats such as JSON. A major challenge here is that the parsing of such data formats is usually highly complex. Accelerating JSON parsing on FPGAs has therefore become a focus of recent research. FPGA accelerators were presented which serve as a co-processor for a CPU to convert JSON into a format that is easier for the CPU to process, e.g., Apache Arrow. However, in case the parsed data should be further processed on the FPGA, such solutions are insufficient as the format created is unsuitable for further processing on FPGAs and, above all, because the accelerators have an immense resource requirement.**

In this paper, we present a novel FPGA parser architecture that is able to interpret JSON data to selectively extract attributes based on a query expression into a format suitable for stream processing on FPGAs. Furthermore, it is shown how the sparsity of JSON can be used to implement a resource-efficient design, only requiring few FPGA resources. This leaves the major share of resources free for accelerating subsequent processing steps of a given application. Our experimental evaluation shows that we can achieve a throughput of 51.1 MB/s per kLUT which is about 3.8 times higher than the throughput per LUT achievable on the most efficient related approach.

***Index Terms*—JSON, FPGA, parsing, architecture**

I. Introduction

In the realm of big data applications, especially in the Internet of Things and Industry 4.0, there is an increasing need to analyze large volumes of data at high rates as well as low latency requirements. Stream processing has emerged as a crucial approach to continuously process and analyze data as it is generated or received. To meet the growing demands for high throughput and low latency in stream processing applications, FPGA accelerators have been proposed as a viable solution in previous studies [15, 20]. As stream applications typically run for long time periods, the relatively long synthesis times required to synthesize application-specific accelerators are tolerable, as they enable perfectly tailored and thus very resource- and energy-efficient data processing.

In order to build such FPGA accelerators, various approaches to compiling queries to FPGA primitives have been presented in the past, see, e.g., [15, 16, 18]. However, these approaches ignore the fact that the data arriving at the FPGA is usually formatted in a way that is difficult to process directly on the FPGA, or even by machines in general. In fact, parsing such data can take most of the time when processed on a CPU. It has been shown in [11] that in many streaming applications, parsing JSON data can account for around 90% of the overall execution time on CPUs.

There are two major benefits to offloading the parsing to an FPGA. First, this would free the CPU from this time-consuming task, allowing it to be better utilized for other tasks or workloads. Second, it would be particularly beneficial if FPGAs could access the data stream directly, e.g., when attached to a network interface or in the form of FPGA-based smart NICs, as additional data movement could be avoided. However, this requires concepts to parse the serialized data format as a byte stream.

Two approaches to parsing JSON data on FPGAs have recently been presented [3, 17]. However, these approaches only consider the acceleration of the parsing process as co-processors of a CPU, and thus only support traditional software-based data processing. Consequently, they parse the received data into a format, respectively data structures that are tailored for further processing on a CPU. In contrast, the approach presented in this paper aims to enable a complete and efficient JSON parsing and processing pipeline in hardware for FPGAs. Our main contributions are:

- Architecture: Presentation of a novel FPGA JSON parser architecture that *(a) enables the selective projection of specific JSON attributes, thereby allowing for further processing of those attributes on the FPGA, (b) features low resource consumption by exploiting the sparsity of JSON data, hence freeing up resources to accelerate subsequent application parts.*
- Design automation: Presentation of a generative approach for *automatically generating application-specific and thus highly resource-efficient parser accelerators*.
- Evaluation: Detailed analysis of the impact of the query expression and data set on the generated parser accelerator in terms of resource requirements and throughput.

II. Related Work

There exists already extensive literature on processing XML data on FPGAs [4, 21, 22]. XML parsing on FPGAs has always been accompanied by path expressions for projections to reduce the amount and complexity of outstream data. Similar to JSON, XML's recursive structure, allowing arbitrary nesting at any point, poses a major challenge. Previous research on XML parsing [12, 13, 14] addressed this problem with pushdown automata, which we now want to introduce for JSON parsing as well. Besides the lower verbosity of JSON compared to XML, our approach differs from previous

1946-1488/23/$31.00 ©2023 IEEE
DOI 10.1109/FPL60245.2023.00034

Authorized licensed use limited to: Danmarks Tekniske Informationscenter. Downloaded on October 05,2026 at 16:51:59 UTC from IEEE Xplore. Restrictions apply.

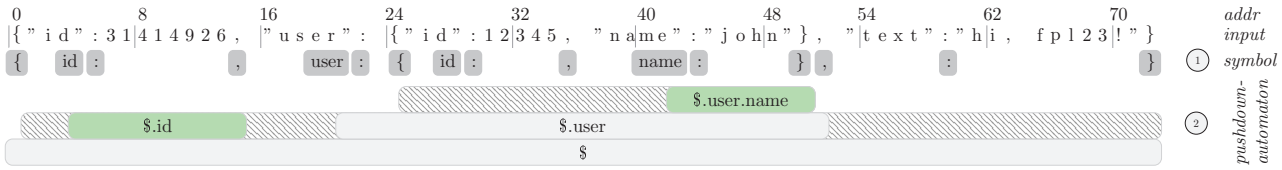


Fig. 1: Parser flow example for extracting attributes *id* and *username* using the JSONPath *$.(id | user.name)*.

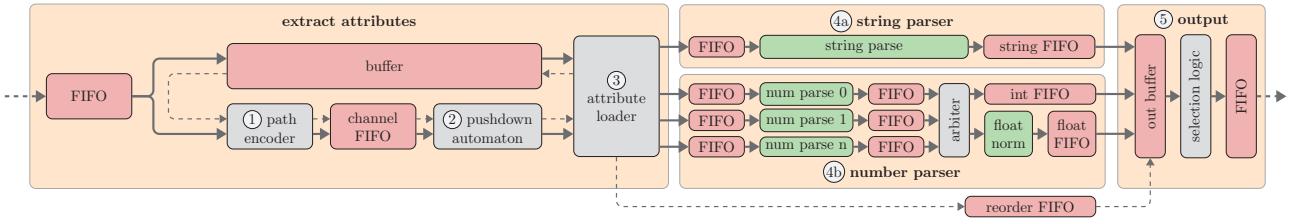


Fig. 2: Overview of the proposed JSON parser architecture.

research by using path expressions not only to filter ASCII data but to project selected attributes using specified data types.

Recently, research attention has shifted to the JSON format which is predominant today, because of its smaller data footprint and better readability. However, research on JSON parsing on FPGAs is still scarce, and furthermore, there is no work known to us on hardware architectures combining JSON parsing with subsequent query processing on the FPGA. Peltenburg et al. [17] propose to speed up JSON parsing by converting the input data to the columnar in-memory format Apache Arrow. The FPGA performs the parsing of the JSON data, converts the data to the Arrow format, and then transmits it to the host memory. The authors implement their parser with a one-to-one relation between fields in the schema and parser blocks, which hence expects the same schema at all times. However, there is no fixed schema for JSON, so valid data in terms of the JSON specification can cause the parser to enter an invalid state. PipeJSON [3] takes a more flexible approach, converting the JSON data into a tape structure that can be efficiently interpreted in software on CPUs. This tape structure consists of a variable-length array of 64-bit values, which contains the decoded values as well as delimiters for identifying array and object structures. While such flexible data structures can be processed efficiently on the CPU, they are rather unsuitable for subsequent stream processing on the FPGA. Hahn et al. [7, 8] present acceleration techniques for raw filtering of JSON data on FPGAs. Raw filtering is a selection technique to optimistically pre-filter data on the raw byte stream of serialized JSON data. However, since only irrelevant JSON records are filtered out and the data is not completely parsed, it does not directly enable any further processing of a given data stream on the FPGA

III. Parser Architecture

The goal of the proposed parser architecture is to parse and directly extract query-related attributes from JSON byte streams at line rate. JSONPath [6] is a query language for specifying the JSON attributes to be selected by using path expressions. The attributes are specified by recursive paths in the JSON structure (e.g., *$.user.id* to select the *user id* in Figure 1). In the following, we propose a generic parser architecture that can be automatically generated based on a given JSONPath expression.

JSON is a context-free grammar and JSON records can be highly nested. While parsing, it is thus necessary to keep track of the current position within the hierarchy of the JSON record. For this, we propose the use of a pushdown automaton. The automaton is configured according to the given JSONPath expression for recognizing when queried attributes appear in the byte stream and triggering the subsequent parsing and projection of these attributes.

For being able to achieve high throughput on an FPGA even at low clock rates, not one but multiple input bytes should be processed per clock cycle. While attribute parsing can be easily parallelized by providing multiple instances of modules for transforming the various attribute types (strings, integers, floats) into binary formats, a major challenge is that pushdown automata are extremely difficult to parallelize.

Our approach therefore first extracts only the information relevant to keeping track of the state of the pushdown automaton and encodes it into symbols. It furthermore exploits the sparsity of JSON data so that even a pushdown automaton with a low degree of parallelism is able to interpret the symbols without leading to overall throughput degradation. In the following, we illustrate this concept by means of a motivational example.

Motivational example: Figure 1 shows an example record, which is fed into the parser with 8 bytes per cycle (see grey vertical lines). The JSONPath query is $.(`id | user.name`*). To track the given paths, at first only structural characters (i.e., { , } , [,] , : , ,) and key strings (i.e., id , user , name) are relevant. This information is extracted from the stream (1) and encoded in symbols. The number of data to be processed is thus significantly reduced (i.e., overall*

Authorized licensed use limited to: Danmarks Tekniske Informationscenter. Downloaded on October 05,2026 at 16:51:59 UTC from IEEE Xplore. Restrictions apply.

72 ASCII characters become 16 symbols, corresponding to a reduction down to 22.2%). The symbols are then passed to a pushdown automaton (2) that keeps track of the current position in the JSON record. Whenever a new hierarchy level is reached in the stream, an uninitialized state is pushed onto the automaton's stack (areas filled with hashed patterns in the figure). Whenever a relevant key symbol is observed in the correct state acc. to the given JSONPath query, the state is initialized correspondingly (areas filled without pattern). Moreover, when this state represents the end of a path of the query (green areas in the figure), the projection of the respective attribute is triggered.

Figure 2 depicts an overview of our proposed parsing architecture. Relevant structure symbols as well as key strings from the specified JSONPath are first extracted in the *path_encoder* (1). In parallel, the same input data is written to the input buffer with ascending addresses. The used address is then bundled together with the corresponding extracted symbols and written into a channel FIFO. With 8 input bytes being processed within one clock cycle, the channel FIFO can accept up to 8 symbols simultaneously. These input symbols are then fed into the pushdown automaton (2), which has a degree of parallelism of two. As long as only a fraction of the input characters are structural characters or key strings, the reduction of 8 input symbols to 2 output symbols in the Channel FIFO poses no throughput restrictions (in the motivational example, this fraction was 22% which is below the bound of 25%).

Now, whenever the pushdown automaton reaches a state that represents an attribute to be projected according to the JSONPath expression, it passes the observed buffer address as well as the *attribute_id* of the attribute to the *attribute_loader*. The *attribute_loader* (3) reads the data from the buffer using the obtained address and writes it to one of the execution input FIFOs based on the attribute type (i.e., number or string). The attributes are then parsed in parallel. The *string parser* (4a) only has to extract the bytes representing characters of the string attribute from the data word. Multiple *number parsers* (4b) are provided over multiple parallel channels, each processing numbers with only one byte per cycle. In the last step (5), the attributes are read from the respective FIFOs in the correct order using the *attribute_ids* received from the *attribute_loader* and written to an attribute-specific address in the output buffer. Finally, selections can be applied on the basis of the buffered attributes, in order to directly discard non-matching records. In the following, the individual stages of the proposed architecture are described in detail.

A. Path encoder

The *path_encoder* first computes bitmasks to mask strings and escaped characters over the next 8 input bytes. All non-masked structural characters are extracted. To extract the key symbols, several string comparators are instantiated, which test in parallel whether a key string starts at each possible word offset (0-7) of the string-masked input.

B. Pushdown automaton

The heart of our proposed parser architecture is the pushdown automaton, which tracks the given path in the observed input record. Its behavior is defined by a finite state machine (FSM) consisting of a list of states and a transition matrix. These are generated acc. to the given JSONPath expression at design time. The following operations can be performed on the stack to track the paths with the pushdown automaton:

a) push: For each opening bracket (i.e., { or [ ), a *push* operation is performed on the stack. A new, uninitialized state is added to the top of stack (TOS) and the *stack pointer (sp)* is incremented, so that it points to the TOS.

b) pop: A *pop* operation is performed upon observing a closing bracket (i.e., } or]). The TOS is reset and the *sp* is decremented.

c) set: When a key symbol (e.g., id) is encountered, the transition matrix is examined for a transition from the previous state (TOS-1) using the observed key. If such a transition exists, the TOS state is set to the identified target state.

d) reset: A comma symbol , causes a reset, whereby the state of the TOS is reset.

e) project: For a project operation, it is first necessary that the current state in the TOS corresponds to the end of a path. If so, the address s_c of the first colon : symbol is stored as the address of the attribute to be projected. At the address s_n of the next following symbol (, or }), the length of the attribute can be determined from the difference s_n - s_c since both symbols delimit the beginning and the end of the attribute. The projection information (i.e., attribute_id, address, length) is passed to the *attribute_loader*.

d) end_of_record: When a *pop* operation is performed on stack level 1 (i.e., moving to the root state $), the end of the record is signaled to the *attribute_loader*.

*Example: Consider again Figure 1 and the JSONPath query $.(id | user.name). Here, an opening bracket { is first encountered, placing an empty state (dashed background) on the stack (**push**). At the symbol id the transition matrix is searched for a transition from TOS-1 (here $) via the symbol id. Since $ has a transition via id to $.id, the TOS is set to the state $.id (**set**). As the state $.id corresponds to the end of a path (green background), a projection is started at the next colon : and the colon address is stored ($s_c = 5$). At the next comma , (at position $s_n = 14$), the length of the attribute is calculated (14-5=9), the projection information is passed to the attribute_loader (**project**) and the TOS is reset (**reset**). With the user symbol, the TOS is set to the $.user state (**set**). As this state does not correspond to a path end, the next colon : can be ignored. With the opening bracket {, an empty state is again placed on top of the stack (**push**). Subsequently, an id symbol is observed again. However, since there is no transition from the $.user state via the id symbol in the transition matrix, no set is performed. For the name symbol on the other hand such an entry exists again so the TOS is set to the $.user.name state (**set**). For this state, a **project** and finally a **pop** are performed using the*

Authorized licensed use limited to: Danmarks Tekniske Informationscenter. Downloaded on October 05,2026 at 16:51:59 UTC from IEEE Xplore. Restrictions apply.

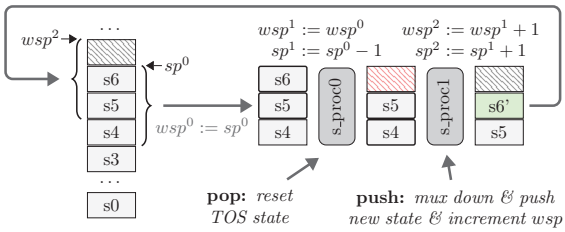


Fig. 3: Conceptual overview of a pushdown automaton that is able to process two input symbols per cycle.

colon : *and closing bracket* } *symbols. With the following comma the* `$.user` *state is reset. Finally, a* ***pop*** *is performed with the last closing bracket* } *and the end of the record is reported to the attribute_loader (****end_of_record****).*

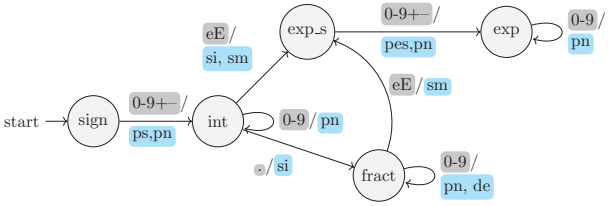
The described logic is implemented in a module (*s_proc*) that can process one input symbol per cycle. To be able to process two symbols per cycle, two of these modules are instantiated in series (see Figure 3). In addition to the input symbol, the TOS state and the TOS-1 state are also passed as *s_proc* module inputs. While the TOS state is needed for almost all operations, the TOS-1 state is only needed to identify valid transitions during the *set* operation. The output of the module then provides the modified TOS and TOS-1 state resulting from the operations described above. With a pop operation, an empty state is output at the TOS output and the stack pointer is decremented. To ensure that the second s_proc module still receives two valid states in the event of a pop operation in the first module, the three most recent states (i.e., TOS, TOS-1, TOS-2) are loaded from the stack at the beginning of each cycle using the *sp*, which always points to the TOS. To be able to write back the local 3 states correctly, an additional *write stack pointer (wsp)* is required. This pointer is set to the *sp* at the beginning of each cycle and incremented for each *push* operation.

C. Execution stage

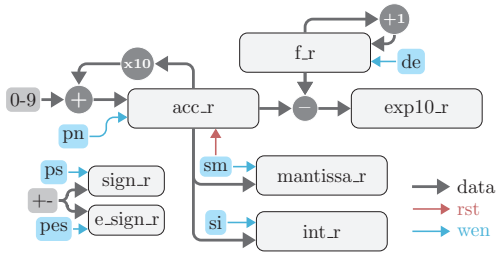
Once the pushdown automaton has found a desired attribute, it is loaded by the attribute loader from the buffer into one of the execution FIFOs depending on its data type.

1) String parse: To parse strings, only a possible word offset has to be compensated. For this purpose, each input word is first written into a register. Then, the offset can be determined by identifying the first '1' in the string bitmask (see Section III-A).

2) Number parse: While parsing numbers is much more complex, here the independent processing of attributes can be used to our advantage. Instead of parallelizing the complex logic at great resource costs, we can simply use multiple parallel channels, each processing one attribute with only 1 byte per cycle. As well as being simpler and more efficient to implement, this has the advantage of allowing us to tailor the number of number parsers to the statistical properties of the query and data.



(a) Control path.



(b) Data path.

Fig. 4: Number parser architecture.
ps/*pes*: parse (exp) sign; *pn*: parse num; *si*: save int; *sm*: save mantissa; *de*: decrement exp;

Our parser supports all numbers of the JSON specification [2]. Numbers are composed of several parts, each of which is mapped to a state of an FSM (see Figure 4) to parse the respective part: an optional sign (*sign*), an integer (*int*), an optional fraction (*fract*), and an optional exponent (*exp*), which in turn may contain an optional sign (*exp_s*). In the data path, as long as the number parser is active (*pn*), for each valid number at the input, the accumulator register (*acc_r*) is multiplied by 10 and the received number is added. If a point . is observed, the FSM enters the *fract* state, whereby the accumulator is written to an integer register (*int_r*), as this corresponds to the integer part of the parsed number. In the *fract* state, the number of decimal places *f_r* is counted to later subtract *f_r* from the exponent to compensate for the extra multiplications by 10. At the transition to the exponent state, the content of the accumulator is stored in the corresponding registers (i.e. *mant_r* and *int_r* when transitioning from *int*), so that the accumulator can be reused to parse the exponent. If the FSM encounters a symbol that is not annotated at the edges, the number parsing is terminated, and depending on the current state, the corresponding registers are written (i.e. *int_r* when in *int* and *mantissa_r* when in *fract*) and the output is set to valid.

Since the output of the individual channels is only active once per parsed number, these can be arbitrated to one channel without impairing the throughput. If the number is to be projected as a float, it is necessary to convert the number with an exponent of base 10 to one with an exponent of base 2. To do this, the integer part is first converted into a float. Furthermore, for determining the exponent of base 2, a BRAM-based lookup is used. The converted integer, as well as the float exponent, can then be simply multiplied to obtain an IEEE754 compliant float.

Authorized licensed use limited to: Danmarks Tekniske Informationscenter. Downloaded on October 05,2026 at 16:51:59 UTC from IEEE Xplore. Restrictions apply.

D. Output Stage

In the output stage, the attributes are finally assembled into the correct order, possible filter expressions are applied, and the data is serialized into the output stream. For this, the output stage receives path-specific information about the projected attributes as well as filter expressions to be performed, which will be discussed in more detail in Section IV. With the help of this attribute list, entries are gradually read from the reorder FIFO, containing the *attribute_id* of the next valid attribute, which is used to write the data from the previous execution stage FIFOs into attribute-specific registers. When an *end of record* entry is read, a valid flag is set and the registers are written to a buffer which is used to perform the selection. If no selections are provided, the data is serialized directly.

As filter expressions, our parser currently supports AND combinations of any number of attribute comparisons. Here, we support direct comparisons of two strings and $<, >, \leq, \geq, =$ comparisons of two integers or two floats. The filter logic is then converted to hardware components (i.e., comparators, *and*-gates) in a one-to-one mapping and the result is *and*-combined with the valid flag. The final serialization is done using an FSM, which outputs the buffer contents word-by-word on the output port.

IV. Parser Generator

The presented parser architecture contains three components whose logic strongly depends on the given JSONPath: the *path encoder* (1), the pushdown automaton (2) and the *output stage* (5). For the path encoder, a dictionary with all necessary key strings and their associated binary code is created (e.g., {"id": 0, {"user": 1, "name": 2} for *$.(id | user.name)*). Based on this dictionary, the corresponding string comparators are instantiated during the parser generation.

Moreover, this dictionary is used to generate the finite automaton consisting of a list of states as well as a transition matrix for the pushdown automaton (see Section III-B). The list of states corresponds to all possible partial paths of the given JSONPath. For associating the respective attribute with states that correspond to the end of a path, an *attribute_id* is assigned in these cases. The list of states of the running example in Figure 1 would thus be: [{*"n":"$"*}, {*"n":"$.id", "attr_id":1*},{*"n":"$.user"*}, ...]. In the transition matrix, the different states are then linked via the entries of the string dictionary (e.g., {*$:{ "id":$.id, "user":$.user }, $.user:{ "name":$.user.name }*}).

Finally, the attribute list for the output stage must be generated. For this purpose, an entry is created for each assigned *attribute_id*. To be able to distinguish between different data types, we have extended the JSONPath specification with typecasts. These are specified after the respective path using two colons, the desired type and the desired length (i.e., *.attr::type(len)*). The length of floats and ints is specified in bits (e.g.,*$.id::int(64)*) whereas the length of strings is specified in bytes (e.g.,*$.text::string(16)*). While for strings arbitrary lengths can be selected, for integers and floats only 32-bit and 64-bit are supported. The buffer offsets are then determined in the order in which the attributes are specified in the JSONPath by adding the length of each attribute to the offset of the following one. In addition, strings of dynamic length are supported (e.g., *$.text::string(*)*), which are buffered separately and output after the static part of the record. The corresponding attribute field consists of a 16-bit offset which points to the start of the string and a 16-bit length field which specifies the length of the string in bytes. If a filter expression is present in the specified path (e.g., *$[?id=42].text*), a filter configuration must be created last. This is a list of comparisons whose entries include the type of comparison, the referenced attribute id, and the given comparison operator. Here we have again extended the JSONPath specification to be able to apply filter expressions to entire records (e.g., *$[?(id=42)]* to select only records with an id equal to 42).

All hardware components were designed in the Python-based hardware description language Amaranth HDL[1]. Consequently, the instantiation and configuration of the components can be achieved by simply instantiating and parameterizing Python classes. Thanks to the Python base, existing Python modules can be used to transform path expressions into tree structures and dictionaries, which in turn can be utilized to generate the parser accelerator as previously described.

V. Evaluation

As resource consumption and the achievable throughput of the presented parser architecture strongly depend on the given JSONPath and its configuration, these relationships will be elaborated in the following. The generated parser accelerators have been synthesized for and evaluated on a Xilinx ZCU106 Zynq SoC.

A. Benchmark evaluation

We evaluated seven path expressions (see Table I) over three different datasets. The first is the *twitter* dataset with very deep nesting and strong variance among the records. The second is the *kostya* dataset, which is composed of a large number of coordinates and thus long floating point numbers. The third and last is, the *taxi* dataset, which is meant to represent a typical IoT scenario and is mainly composed of integers and floats. The JSONPaths used for this data are listed in Table I. The paths for the *twitter* and *kostya* datasets were taken from On-Demand JSON [10] and the *taxi* path is based on the RIoTBench [19].

The bottlenecks of our architecture are the following: the *a) pushdown automaton* can only process up to two structural/ key symbols per cycle. The *b) number parsers* can process one number character per instance and cycle. The *c) attribute_loader* needs at least two cycles for each projected attribute. Table II lists the percentage of structural (*struct.*) and key symbols as well as projected numbers (*projected num.*) per benchmark. The share of structural/ key symbols is 15.1% in the *kostya* benchmark and even less than 1/8 = 12.5% in all

[1]Amaranth HDL: https://github.com/amaranth-lang/amaranth

Authorized licensed use limited to: Danmarks Tekniske Informationscenter. Downloaded on October 05,2026 at 16:51:59 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Benchmark JSONPaths.

Bench	JSONPath
tw0	$.((user.id::int(64)) | (user.created_at::string(32)) | (retweet_count::int(64)) | (favorite_count::int(64)) | (reply_status_id::int(64)) | (text::string(*)))
tw1	$.((user.id::int(64)) | (retweeted_status.id::int(64)))
tw2	$[?(id = 505874901689851904)].((id::int(64)) | (text::string(*)))
tw3	$.((user.screen_name::string(32)) | (retweet_count::int(64)) | (text::string(*)))
ko0	$.((x::float(32)) | (y::float(32)) | (z::float(32)))
ta0	$.((trip_time_in_secs::int(64)) | (tip_amount::float(32)) | (fare_amount::float(32)) | (tolls_amount::float(32)) | (trip_distance::float(32)))
ta1	$[?(trip_time_in_secs>=140 & trip_time_in_secs<=3155 & tip_amount>=0.65 & tip_amount<=38.55 & fare_amount>=6.0 & fare_amount<=201 & tolls_amount>=2.5 & tolls_amount<=18 & trip_distance>=1.37 & trip_distance<=29.68)].((trip_time_in_secs::int(64)) | (tip_amount::float(32)) | (fare_amount::float(32)) | (tolls_amount::float(32)) | (trip_distance::float(32)))

other benchmarks, thus below the maximum throughput of the pushdown automaton (i.e. 2/8 = 25%). For all except *kostya*, the percentage of projected numbers is below 12.5% thus a single instance of a number parser suffices to sustain maximum throughput. In contrast, the *kostya* benchmark contains 46.5% of projected numbers and requires four instances to maintain the maximum achievable throughput (i.e. 4/8 = 50%).

Table II furthermore depicts the resources required for each path, as well as the maximum achievable clock rate and throughput achieved in our experiments (see next section). The resource consumption depends primarily on the number and size of the attributes to be projected. While only a few attributes are projected in the *kostya* path in the absence of additional nesting, the resource requirements are somewhat higher due to the higher number of number parsers (4 instead of 1).

Finally, it can be seen from ta0 and ta1 that the additional filters only slightly increase the resource requirements. On the other hand, thanks to the consistent pipelining, the maximum clock speed achieved in the benchmarks examined is fairly independent of the path, allowing different paths in the FPGA to be swapped without adjusting the clock speed.

B. System Architecture

Figure 5 depicts the overall system architecture, based on [1], consisting of a tightly coupled ARM CPU and programmable logic (PL) into which our parser design has been integrated and tested. The PL contains several dynamically Reconfigurable Regions (RRs), which are connected to each other and to various interfaces via a crossbar. In each of the RRs resides a DMA engine, managed by the on-chip ARM CPU.

In our experiments, 2.3 MB of JSON data from the *twitter* dataset, 2.4 MB MB from the *kostya* dataset, as well as 1.6 MB from the taxi dataset, were preloaded into the RAM and transferred to the parser accelerator (blue RR in the figure) using DMA. The results containing the parsed JSON objects were

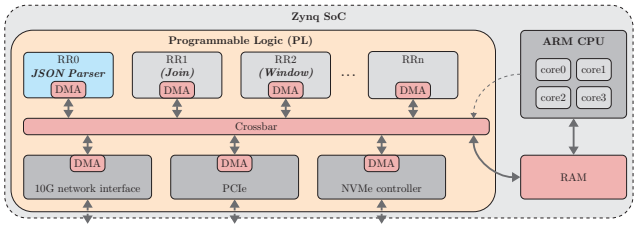


Fig. 5: SoC system architecture used for evaluation.

again written back to RAM via DMA. For the experiments, the entire system was clocked at 200 MHz, thus with a word width of 64 bits, a maximum throughput of 1.6 GB/s should be achievable in the ideal case. In practice, however, we only achieved a maximum throughput of 1.45 GB/s, which was due to the CPU not being able to schedule new DMA descriptors fast enough. In the future, this problem could be solved by using more cores to schedule the DMAs, or even by adding a dedicated hardware component for scheduling the DMAs. For us, however, the achieved throughput is sufficient since it suffices to process the incoming data from a 10 GBit/s network interface at line rate. Our system architecture can thus be used to process and accelerate further stages (grey RRs in the figure) or even an entire data stream processing application on the FPGA.

With the exception of the *kostya* benchmark, the 1.45 GB/s could be achieved with all benchmarks. In the Kostya benchmark, only a throughput of 1.29 GB/s was achieved, which can be attributed to a bottleneck in the *attribute_loader* caused by the overall high ratio of projected data to input data. The *attribute_loader* is on the one hand implemented in such a way that between each loaded attribute, an empty cycle must be kept, on the other hand, the loaded attributes are usually not aligned on word boundaries, with which additionally irrelevant characters are loaded. While the first problem could be solved by minor design changes, the second problem would require increasing the memory bandwidth and thus the word size of the memory read port. State-of-the-art CPU parsers [9] achieve 1.6-3.7 GB/s on the evaluated data sets (see Table II).

Finally, we compared our overall system consisting of a parser accelerator, a hardware Ethernet/IP stack, and DMA engines against the related work (see Table III). While PipeJSON [3] works schema independent, for the evaluation of JSON-to-Arrow [17] a complex and a simple schema were used which are not available to us. We have therefore also chosen a complex (*tw0*) and a simple schema (*tw1*) from our benchmarks for this comparison. However, potential differences between our schemas and those in related work will not strongly affect the comparison, since in our approach the path-related variance is small compared to the additional resource requirements of the Ethernet/IP stack and the DMA engines (i.e., the complex parser accounts for only about a quarter of all required LUTs). Since we only targeted the 10 GBit/s of the network interface, our maximum achievable throughput is considerably lower. However, our design is significantly more

Authorized licensed use limited to: Danmarks Tekniske Informationscenter. Downloaded on October 05,2026 at 16:51:59 UTC from IEEE Xplore. Restrictions apply.

TABLE II: Benchmark results.

bench-mark	dataset evaluation				FPGA (SPEAR-JSON)						CPU [9]
	symbol occurrence			projected num.	resources				max freq. [MHz]	throughput [GB/s]	throughput [GB/s]
	struct.	key	all		LUTs	FFs	BRAMs	DSPs			
tw0	5.1%	0.3%	5.4%	0.3%	6877 (3.0%)	2903 (0.6%)	2 (0.6%)	2 (0.1%)	303	1.46	3.2-3.4
tw1	5.1%	0.1%	5.2%	0.4%	4230 (1.8%)	1625 (0.4%)	2 (0.6%)	2 (0.1%)	322	1.46	3.2-3.5
tw2	5.1%	0.1%	5.2%	0.3%	4797 (2.1%)	1870 (0.4%)	2 (0.6%)	2 (0.1%)	330	1.46	3.4-3.7
tw3	5.1%	0.1%	5.2%	0.0%	5950 (2.6%)	2399 (0.5%)	2 (0.6%)	2 (0.1%)	331	1.46	3.3-3.6
ko0	12.8%	2.3%	15.1%	46.5%	5793 (2.5%)	2526 (0.5%)	2 (0.6%)	2 (0.1%)	298	1.29	1.6-1.8
ta0	8.6%	1.0%	9.6%	3.5%	5531 (2.4%)	1825 (0.4%)	2 (0.6%)	2 (0.1%)	314	1.46	n.a.
ta1	8.6%	1.0%	9.6%	3.5%	5586 (2.4%)	1992 (0.4%)	2 (0.6%)	2 (0.1%)	303	1.46	n.a.

TABLE III: Comparison of throughput, resource requirements and resource efficiency to related work.

	SPEAR-JSON		JSON-to-Arrow [17]		PipeJSON [3]
	simple	complex	simple	complex	
throughput [GB/s]	1.46	1.46	18.37	10.10	19.33
resources [kLUTs]	26	29	1,348	1,566	1,326
resource efficiency [MBs/kLUTs]	56.5	51.1	13.6	6.4	14.6

resource-efficient in terms of achievable throughput per LUT.

In addition, we evaluated the expected resource efficiency for a 100G setup using the *Corundum* SmartNIC framework [5] on an Alveo U280 board to demonstrate the scalability of our approach. The *Corundum* framework allows the parser accelerators to be inserted directly into the data stream of incoming network packets. To saturate the 100 GBit/s, eight of the presented parser instances are instantiated in parallel, i.e., at a clock rate of 200 MHz, allowing for a throughput of 12.8 GB/s (95.4 GBit/s). A round-robin *splitter* component is needed to distribute the incoming packets among the eight accelerators, and an *arbiter* is needed to merge the eight accelerator outputs. The total resource requirements are 82.1 kLUTs *(Corundum)* + 3.5 kLUTs *(splitter & arbiter)* + $8 \cdot 6.8$ kLUTs *(tw0 parsers)* = 140 kLUTs. With a throughput of 12.8 GB/s, this results in a resource efficiency of 91.4 MBs/kLUTs. While this number is based on a resource estimate, a lot of resources can be saved by using the JSON parser in such a pipelined streaming setup (i.e., no extra DMA engines, no big interconnect).

VI. Conclusion

The sparsity of JSON can be used to create a highly resource-efficient FPGA design for parsing and extracting only the relevant parts of the input data. Path expressions can be used to specify which attributes (i.e., input data parts) are of interest, resulting in a consistent layout of the output data. This ultimately enables further processing of the parsed data on the FPGA. Here, we proposed a lean design requiring only very few FPGA resources. The remaining resources can be used to accelerate subsequent parts of the stream processing application. Our evaluation showed that the designs generated for all examined benchmarks can process a stream of incoming data from a 10 GBit/s interface at line rate, making our approach perfectly suited for deployment on, e.g., a SmartNIC that processes data directly at its source.

In the future, we want to consider designing the path-specific components in such a way that they can be programmed online (i.e. can be changed without reconfiguration). Moreover, we would like to implement the output stage in an instruction set-based, CPU-like manner to allow for more complex filter expressions (i.e., array selections).

References

[1] A. Becher, A. Herrmann, S. Wildermann, and J. Teich. Reprovide: Towards utilizing heterogeneous partially reconfigurable architectures for near-memory data processing. Gesellschaft für Informatik, Bonn, 2019. doi: 10.18420/btw2019-ws-04. URL https://doi.org/10.18420/btw2019-ws-04.

[2] T. Bray. The JavaScript Object Notation (JSON) Data Interchange Format. RFC 8259, Dec. 2017. URL https://www.rfc-editor.org/info/rfc8259.

[3] J. Dann, R. Wagner, D. Ritter, C. Faerber, and H. Froening. PipeJSON: Parsing JSON at line speed on FPGAs. DaMoN'22, 2022. URL https://doi.org/10.1145/3533737.3535094.

[4] F. El-Hassan and D. Ionescu. A hardware architecture of an XML/XPath broker for content-based publish/subscribe systems. In *2010 International Conference on Reconfigurable Computing and FPGAs*, pages 138–143. IEEE, 2010.

[5] A. Forencich, A. C. Snoeren, G. Porter, and G. Papen. Corundum: An open-source 100-Gbps NIC. In *28th IEEE International Symposium on Field-Programmable Custom Computing Machines*, 2020.

[6] S. Gössner. *JSONPath - XPath for JSON*. URL https://goessner.net/articles/JsonPath/.

[7] T. Hahn, A. Becher, S. Wildermann, and J. Teich. Raw Filtering of JSON data on FPGAs. In *DATE'22*, 2022. doi: 10.23919/DATE54114.2022.9774696. URL https://doi.org/10.23919/DATE54114.2022.9774696.

[8] T. Hahn, S. Wildermann, and J. Teich. Auto-tuning of raw filters for FPGAs. In *2022 32nd International Conference on Field-Programmable Logic and Applications (FPL)*, pages 167–175, 2022. doi: 10.1109/FPL57034.2022.00036.

Authorized licensed use limited to: Danmarks Tekniske Informationscenter. Downloaded on October 05,2026 at 16:51:59 UTC from IEEE Xplore. Restrictions apply.

[9] G. Langdale and D. Lemire. Parsing gigabytes of JSON per second. *The VLDB Journal*, 28(6), Oct 2019. doi: 10.1007/s00778-019-00578-5.

[10] D. Lemire and J. Keiser. On-Demand JSON: A Better Way to Parse Documents? Jan 2023. URL https://doi.org/10.22541/au.167285898.80392303/v1.

[11] Y. Li, N. R. Katsipoulakis, B. Chandramouli, J. Goldstein, and D. Kossmann. Mison: A fast JSON parser for data analytics. *Proc. VLDB Endow.*, 10(10):1118–1129, 2017. doi: 10.14778/3115404.3115416. URL http://www.vldb.org/pvldb/vol10/p1118-li.pdf.

[12] A. Mitra, M. Vieira, P. Bakalov, W. Najjar, and V. Tsotras. Boosting XML filtering with a scalable FPGA-based architecture. 2009. URL https://arxiv.org/abs/0909.1781.

[13] R. Moussalli, M. Salloum, W. Najjar, and V. Tsotras. Accelerating XML query matching through custom stack generation on FPGAs. In *High Performance Embedded Architectures and Compilers: 5th International Conference, HiPEAC 2010, Pisa, Italy, January 25-27, 2010. Proceedings 5*, pages 141–155. Springer, 2010.

[14] R. Moussalli, M. Salloum, W. Najjar, and V. J. Tsotras. Massively parallel XML twig filtering using dynamic programming on FPGAs. In *2011 IEEE 27th International Conference on Data Engineering*, pages 948–959. IEEE, 2011.

[15] R. Mueller, J. Teubner, and G. Alonso. Streams on wires: A query compiler for FPGAs. Aug. 2009. doi: 10.14778/1687627.1687654. URL https://doi.org/10.14778/1687627.1687654.

[16] R. Mueller, J. Teubner, and G. Alonso. Glacier: a query-to-hardware compiler. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*, pages 1159–1162, 2010. URL https://doi.org/10.1145/1807167.1807307.

[17] J. Peltenburg, A. Hadnagy, M. Brobbel, R. Morrow, and Z. Al-Ars. Tens of gigabytes per second JSON-to-Arrow conversion with fpga accelerators. In *ICFPT'21*, 2021. doi: 10.1109/ICFPT52863.2021.9609833. URL https://doi.org/10.1109/ICFPT52863.2021.9609833.

[18] M. Sadoghi, R. Javed, N. Tarafdar, H. Singh, R. Palaniappan, and H.-A. Jacobsen. Multi-query stream processing on FPGAs. In *2012 IEEE 28th International Conference on Data Engineering*, pages 1229–1232, 2012. doi: 10.1109/ICDE.2012.39. URL https://doi.org/10.1109/ICDE.2012.39.

[19] A. Shukla, S. Chaturvedi, and Y. Simmhan. Riotbench: A real-time IoT benchmark for distributed stream processing platforms. *arXiv preprint arXiv:1701.08530*, 2017.

[20] J. Teubner and R. Mueller. How soccer players would do stream joins. SIGMOD'11, 2011. URL https://doi.org/10.1145/1989323.1989389.

[21] J. Teubner, L. Woods, and C. Nie. Skeleton automata for FPGAs: Reconfiguring without reconstructing. SIGMOD '12, 2012. URL https://doi.org/10.1145/2213836.2213863.

[22] L. Woods and G. Alonso. Fast data analytics with FPGAs. In *2011 IEEE 27th International Conference on Data Engineering Workshops*, pages 296–299. IEEE, 2011. doi: 10.1109/ICDEW.2011.5767669. URL https://doi.org/10.1109/ICDEW.2011.5767669.

Authorized licensed use limited to: Danmarks Tekniske Informationscenter. Downloaded on October 05,2026 at 16:51:59 UTC from IEEE Xplore. Restrictions apply.